Manipal School of Information Sciences

Linux Operating System Lab Assignment

Bash File Investigation and Pattern Analysis
grep, Regular Expressions and File Operators

Student Name: Ribhav Devprasad
Reg. No.: 261100690002

Bash File Investigation Lab

Part A — Basic grep

Exercise 1 — Search File Contents

Use payload.sh as the input file. The following commands answer each requested search.

```
grep 'wget' payload.sh
grep 'nc' payload.sh
grep 'chmod' payload.sh
grep -i 'WGET' payload.sh
grep -n '/tmp' payload.sh
grep -c '/tmp' payload.sh
grep -o 'wget' payload.sh
grep -w 'nc' payload.sh
```

Exercise 2 — Compare Files

Search several files in one command by supplying multiple filenames:

```
grep -n 'wget' payload.sh update.sh cleanup.sh
grep -n '/tmp' payload.sh update.sh cleanup.sh
```

Only payload.sh contains wget. The files update.sh and cleanup.sh do not contain it. The /tmp pattern appears in payload.sh and cleanup.sh, but not in update.sh.

Exercise 3 — Inverse Search

```
grep -v 'INFO' system.log
grep -v 'ERROR' system.log
grep -v 'WARNING' system.log
grep -vc 'INFO' system.log
```

The first command prints every non-INFO entry, the second excludes ERROR entries, and the third excludes WARNING entries. The final command reports **6** non-INFO lines because the log contains five INFO records out of eleven total records.

Part B — Security Log Analysis

Exercise 4 — Analyze system.log

```
grep 'ERROR' system.log
grep 'WARNING' system.log
grep 'FAILED' system.log
grep -c 'ERROR' system.log
grep -n 'WARNING' system.log
grep -i 'error' system.log
grep -q 'Malware' system.log; echo $?
```

There are two ERROR entries, two WARNING entries and one FAILED entry. The warning entries are on lines 3 and 7. Because Malware does not occur, grep -q returns status 1 a zero status would mean that the pattern was found.

Exercise 5 — Context Investigation

```
grep -A 2 'ERROR' system.log
grep -B 2 'ERROR' system.log
grep -B 1 -A 1 'ERROR' system.log
grep -C 2 'FAILED' system.log
```

Part C — egrep / grep -E

Exercise 6 — Search Multiple Security Events

```
grep -E 'ERROR|WARNING|FAILED' system.log
```

This returns five lines: two ERROR records, two WARNING records and the single FAILED record.

Exercise 7 — Multiple Suspicious Commands

```
grep -E 'wget|curl|nc|chmod' payload.sh
```

The patterns found are wget, nc and chmod. The pattern curl is absent. Three lines contain at least one selected pattern.

Exercise 8 — Search Multiple Files

```
grep -l -E 'wget|curl|nc|chmod' *.sh
grep -n -E 'wget|curl|nc|chmod' *.sh
grep -c -E 'wget|curl|nc|chmod' *.sh
```

Only payload.sh contains a selected command. The filename-only form is -l, line numbers use -n, and -c reports the number of matching lines for each file.

Part D — patterns.txt

Exercise 9 — Pattern File Investigation

Use -f to treat each line in patterns.txt as a separate search pattern:

```
grep -f patterns.txt payload.sh
grep -f patterns.txt system.log
grep -f patterns.txt config.conf
grep -f patterns.txt *.sh
grep -l -f patterns.txt *
grep -h -f patterns.txt payload.sh system.log config.conf
```

The indicator list matches wget, nc and chmod in payload.sh ERROR, WARNING and FAILED in system.log and password in config.conf.

Part E — Regular Expressions Using filelist.txt

Exercise 10 — Beginning and End of Line

```
grep '^malware' filelist.txt
grep '^script' filelist.txt
grep '\.sh$' filelist.txt
grep '\.py$' filelist.txt
grep '\.log$' filelist.txt
grep '^p.*\.sh$' filelist.txt
```

The results are, respectively: the three malware files; the two script files; the five shell scripts; the three Python files; the two log files; and payload.sh.

Exercise 11 — Character Classes

```
grep '^malware[12]\.sh$' filelist.txt
grep '[[:digit:]]' filelist.txt
grep '[12]' filelist.txt
grep '^[mp]' filelist.txt
grep '^[su]' filelist.txt
```

The first expression selects malware1.sh and malware2.sh. Character classes can identify any digit, the digits 1 or 2, filenames beginning with m or p, and filenames beginning with s or u.

Exercise 12 — Negated Character Classes

```
grep '^[^m]' filelist.txt
grep '^[^mp]' filelist.txt
grep '^malware[^0-9]' filelist.txt
```

The first command excludes names beginning with m. The second excludes names beginning with either m or p. The third identifies malwareA.sh, because the character after malware is a letter rather than a digit.

Part F — File Operators

Exercise 13 — Test Whether a File Exists

A reusable Bash script is:

```
#!/bin/bash
file=$1
if [ -e "$file" ]; then
  echo "$file exists"
else
  echo "$file does not exist"
fi
```

With the supplied dataset, payload.sh and report.txt exist, while unknown.sh does not.

Exercise 14 — Classify a Path

To distinguish a regular file, directory or missing path:

```
if [ -f "$1" ]; then
  echo "Regular file"
elif [ -d "$1" ]; then
  echo "Directory"
else
  echo "Neither"
fi
```

The expected classifications are: payload.sh and system.log regular files; quarantine a directory; backup a directory; and unknown neither.

Exercise 15 — Permission Tests

```
file=$1
echo "File: $file"
[ -r "$file" ] && echo 'Readable   : Yes' || echo 'Readable   : No'
[ -w "$file" ] && echo 'Writable   : Yes' || echo 'Writable   : No'
[ -x "$file" ] && echo 'Executable : Yes' || echo 'Executable : No'
```

Run the script as a normal user. The result depends on the extracted permissions; in the supplied dataset, the shell scripts are intended to be executable, while report.txt and config.conf are regular non-executable files.

Exercise 16 — Detect Empty Files

```
if [ -s "$1" ]; then
  echo 'File contains data'
else
  echo 'File is empty'
fi
```

payload.sh contains data, whereas empty.txt is empty.

Part G — File Validation with [ ]

Exercise 17 — Validate a File

```
file=$1
if [ ! -e "$file" ]; then
  echo 'File does not exist'
elif [ ! -f "$file" ]; then
  echo 'Path is not a regular file'
elif [ ! -r "$file" ]; then
  echo 'File is not readable'
elif [ ! -s "$file" ]; then
  echo 'File is empty'
else
  echo 'File exists, is regular, readable and non-empty'
fi
```

Exercise 18 — Combine [ ] with grep

```
file=$1
if [ ! -e "$file" ] || [ ! -f "$file" ] || [ ! -r "$file" ] || [ ! -s "$file" ]; then
  echo 'File failed validation'
elif grep -q 'wget' "$file"; then
  echo 'Selected indicator detected — investigate further'
else
```

```
  echo 'Selected indicator not detected'
fi
```

For payload.sh, the search detects wget. For update.sh, validation succeeds but the selected indicator is not detected.

Part H — Tests with [[ ]]

Exercise 19 — Repeat File Tests Using [[ ]]

The same validation can be expressed with Bash's extended conditional syntax:

```
if [[ ! -e $file ]]; then
  echo 'Missing path'
elif [[ ! -f $file ]]; then
  echo 'Not a regular file'
elif [[ ! -r $file ]]; then
  echo 'Not readable'
elif [[ ! -s $file ]]; then
  echo 'Empty file'
else
  echo 'Valid non-empty readable file'
fi
```

Exercise 20 — Identify .sh Files

```
if [[ "$filename" =~ \.sh$ ]]; then
  echo 'Shell script identified for inspection'
else
  echo 'File does not have a .sh extension'
fi
```

The expected result is positive for payload.sh, and negative for scanner.py and report.txt.

Exercise 21 — Identify Several Script Extensions

```
if [[ "$filename" =~ \.(sh|py|pl)$ ]]; then
  echo 'Script file — inspect if required'
else
  echo 'Other file type'
fi
```

This identifies payload.sh and scanner.py as script files, while report.txt and config.conf are not classified by this rule.

Part I — grep, File Operators and Constructs

Exercise 22 — Analyze Executable File Content

```
file=$1
if [[ -e $file && -f $file && -x $file ]]; then
  if grep -q 'wget' "$file"; then
    echo 'Executable file contains wget'
  else
    echo 'Executable file does not contain wget'
  fi
else
  echo 'File is missing, not regular, or not executable'
fi
```

The supplied payload.sh is the important positive test. update.sh should be checked according to its actual executable permission, and report.txt should fail the executable-file condition.

Exercise 23 — egrep with File Operators

```
if [[ -f $file && -r $file && -s $file ]]; then
  if grep -Eq 'wget|curl|nc|chmod' "$file"; then
    echo 'One or more selected indicators detected'
  else
    echo 'Selected indicators not detected'
  fi
else
  echo 'File failed regular/readable/non-empty checks'
fi
```

This detects selected indicators in payload.sh. The same test can be applied safely to every regular file before content is searched.

Part J — Configuration File Investigation

Exercise 24 — Search for Sensitive Configuration Data

```
if [[ -e config.conf && -f config.conf && -r config.conf && -s config.conf ]]; then
  grep -E 'password|username' config.conf
else
  echo 'Configuration file failed validation'
fi
```

The matching lines are username=admin and password=LabPassword123. Configuration files containing authentication information require additional protection because disclosure can expose credentials, enable unauthorized access and assist lateral movement. Permissions, storage, transmission and logging of such files should therefore be controlled.

Part K — Suspicious File Triage

Exercise 25 — file_check.sh

```
#!/bin/bash
read -p 'Enter filename: ' file
echo
echo '----- FILE INVESTIGATION -----'
echo
exists='No'; regular='No'; directory='No'; readable='No'; writable='No'; executable='No'; nonempty='No'; shell='No'
[[ -e $file ]] && exists='Yes'
[[ -f $file ]] && regular='Yes'
[[ -d $file ]] && directory='Yes'
[[ -r $file ]] && readable='Yes'
[[ -w $file ]] && writable='Yes'
[[ -x $file ]] && executable='Yes'
[[ -s $file ]] && nonempty='Yes'
[[ $file =~ \.sh$ ]] && shell='Yes'
printf 'Exists        : %s\n' "$exists"
printf 'Regular File  : %s\n' "$regular"
printf 'Directory     : %s\n' "$directory"
printf 'Readable      : %s\n' "$readable"
printf 'Writable      : %s\n' "$writable"
printf 'Executable    : %s\n' "$executable"
printf 'Non-empty     : %s\n' "$nonempty"
```

```
printf 'Shell Script  : %s\n' "$shell"
echo
if [[ $readable == Yes && $nonempty == Yes ]]; then
  grep -q 'wget' "$file"; wget_status=$?
  grep -Eq 'wget|curl|nc|chmod' "$file"; indicator_status=$?
  [[ $wget_status -eq 0 ]] && echo 'wget          : Found' || echo 'wget          : Not found'
  [[ $indicator_status -eq 0 ]] && echo 'Selected indicators : Found' || echo 'Selected indicators : Not found'
else
  echo 'Content search skipped: file is not readable and non-empty'
fi
echo
echo 'Further inspection recommended'
```

For an input of payload.sh, the summary should show that it exists, is a regular readable, writable, executable and non-empty shell script. wget and the selected-indicator expression are both found. The final message is deliberately cautious: an indicator is a reason for further inspection, not a conclusion that the file is malicious.